Команда Сбера

От API языковой модели к LangChain

Тращенков Сергей

Разработка и внедрение AI-агентов · МФТИ

Материалы

Telegram-канал

Поместятся ли 28 студентов в А-101?



Материалы

Telegram-канал

Маршрут занятия



Материалы

Telegram-канал

Установка: четыре шага и три терминала

Команда Сбера

1. `git clone …/mipt_llm_api`
   материалы занятия
2. `uv sync`
   Python 3.12 и библиотеки из `uv.lock`
3. `.env` ← ключ авторизации
   личный кабинет → проект GigaChat API → «Настройки API» → «Получить ключ»
4. `uv run tools/check_env.py`
   проверка: пакеты, ключ, токен, прокси

ТЕРМИНАЛ 1 · Jupyter

```
$ uv run jupyter lab llm_api_to_langchain.ipynb
```

→ ноутбук

ТЕРМИНАЛ 2 · прокси GPT2GIGA

```
$ uv run tools/run_proxy.py
```

→ 127.0.0.1:8090

ТЕРМИНАЛ 3 · по желанию, до прокси

```
$ uv run tools/start_mitmproxy.py
```

→ :8080, интерфейс :8081

§ 1.1–1.2

Материалы

Telegram-канал

Авторизация: ключ → токен → запрос



§ 1.3–1.4

Материалы

Telegram-канал

Запрос: v2 и v1

Команда Сбера

v2 · POST /v2/chat/completions

```
Authorization: Bearer <токен>
X-Session-ID: <uuid>      # по желанию

{
  "model": "GigaChat-3-Ultra",
  "messages": [{"role": "user",
    "content": [{"text": "Поместятся…"}]}],
  "model_options": {"temperature": 0.7,
                    "max_tokens": 512},
  "tools": [{"functions": {"specifications": […]}}]
}
```

v1 · POST /v1/chat/completions

```
Authorization: Bearer <токен>
X-Session-ID: <uuid>      # по желанию

{
  "model": "GigaChat-3-Ultra",
  "messages": [{"role": "user",
    "content": "Поместятся…"}],
  "temperature": 0.7,
  "max_tokens": 512,
  "functions": […]
}
```

- текст: список частей (v2) · строка (v1)
- функции: `tools` (v2) · `functions` (v1)
- параметры: в `model_options` (v2) · на верхнем уровне (v1)
- заголовки одинаковые

§ 2.1–2.3

Материалы

Telegram-канал

Ответ: v2 и v1

Команда Сбера

v2 · ответ

```
{
  "model": "GigaChat-3-Ultra:32.9.23.6",
  "created_at": 1790809295,
  "messages": [{"role": "assistant",
    "content": [{"text": "Приложению API…"}]}],
  "finish_reason": "stop",
  "usage": {"input_tokens": 23,
    "output_tokens": 50, "total_tokens": 73,
    "input_tokens_details": {"cached_tokens": 0}}
}
x-request-id, x-session-id → SDK: x_headers
```

v1 · ответ

```
{
  "choices": [{"index": 0,
    "message": {"role": "assistant",
                "content": "Приложению API…"},
    "finish_reason": "stop"}],
  "created": 1790809296,
  "model": "GigaChat-3-Ultra:32.9.23.6",
  "object": "chat.completions",
  "usage": {"prompt_tokens": 23, "completion_tokens": 51,
    "total_tokens": 74, "precached_prompt_tokens": 0}
}
x-request-id, x-session-id → SDK: x_headers
```

текст ответа · почему остановилась · расход токенов · из HTTP-заголовков: поиск в журналах, сессия и кэш

§ 2.2–2.3

Материалы · Telegram-канал

Ловушки v2

`"model_options": {"max_tokens": 10}`	→	`finish_reason: length · output_tokens: 10`
		лимит сработал
`"max_tokens": 10` на верхнем уровне	→	`finish_reason: stop · output_tokens: 1047`
		молча проигнорирован
`"content": "Привет"` в сыром HTTP	→	`400` «invalid JSON syntax»
		SDK сам делает список частей
`"tool_config": {"mode": "forsed"}`	→	работает как `auto`, без ошибки
		опечатка не видна

§ 2.4 · 7.6

Материалы

Telegram-канал

Как модель выбирает следующий токен

«Рыжий кот …» — какое слово следующее?



вероятности иллюстративные

§ 3.1

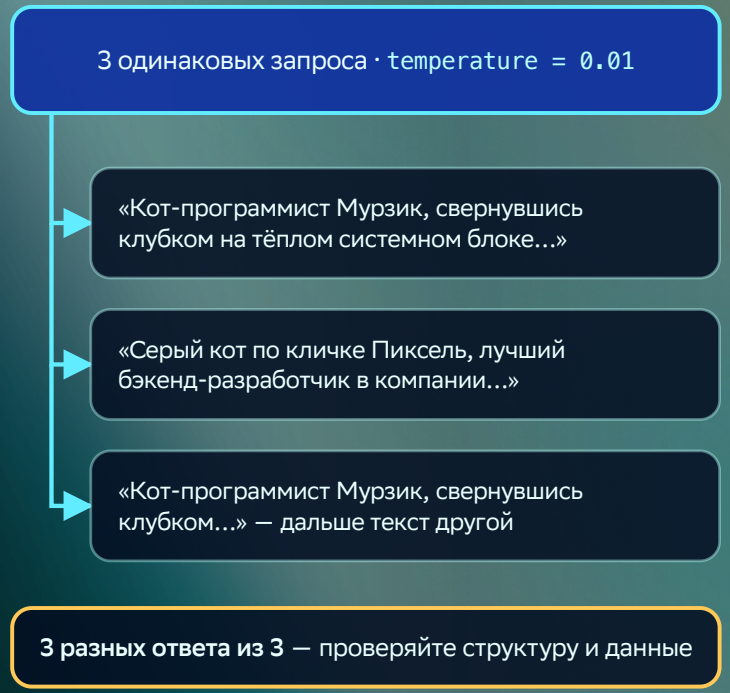
Материалы

Telegram-канал

Параметры и воспроизводимость

v1 — на верхнем уровне тела · v2 — внутри `model_options` · seed нет

`temperature`	случайность выбора	> 0
`top_p`	доля вероятности для выбора	0…1
`max_tokens`	предел длины → `finish_reason: length`	> 0
`repetition_penalty`	штраф за повторы, 1 — без штрафа	> 0
`stream`	ответ частями	true / false
`response_format`	ответ по JSON Schema	схема



§ 3.2–3.3

Материалы

Telegram-канал

Кэш и стоимость

Команда Сбера

запрос = длинный регламент (system) + короткий вопрос

без `X-Session-ID`

каждый запрос — новая сессия

регламент · 960 токенов	30	запрос 1
регламент · 960 токенов	30	запрос 2 — снова целиком

один `X-Session-ID`

`gigachat.session_id_cvar`

регламент · 960 токенов	30	запрос 1
регламент · 960 токенов из кэша	30	запрос 2: `input_tokens: 30` `cached_tokens: 960`

оплачиваются `input_tokens` + `output_tokens`; токены из кэша — нет · `input_tokens` их не включает

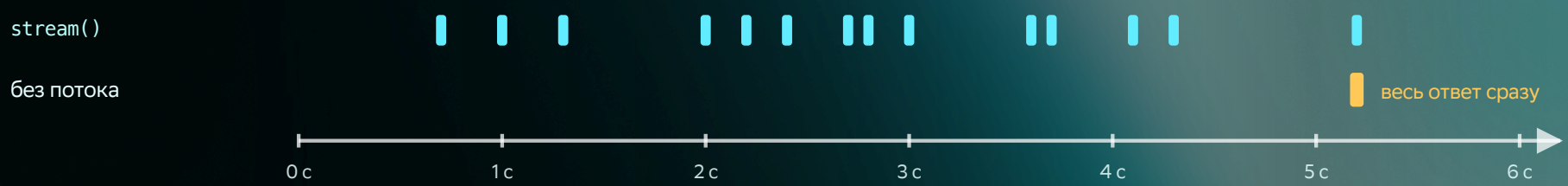
Материалы

Telegram-канал

Другие методы API



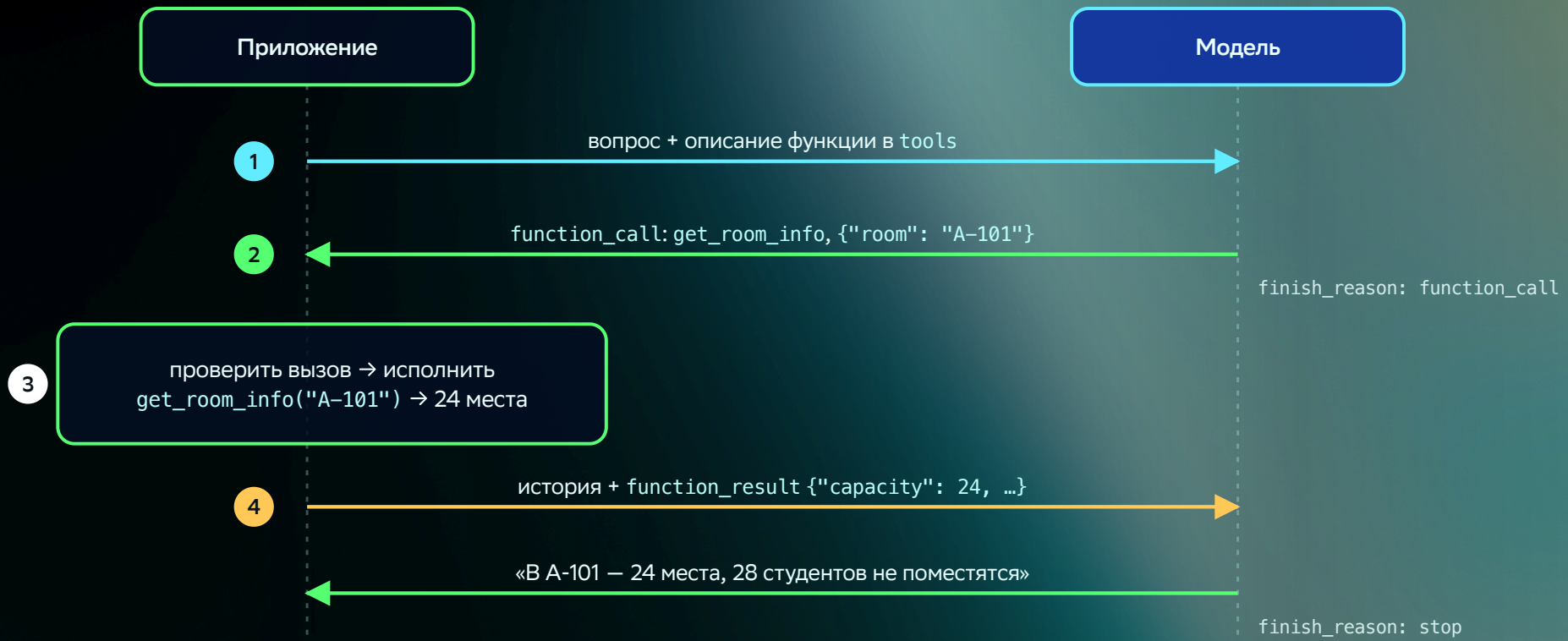
потоковая выдача: длинный ответ, когда приходят части



Материалы

Telegram-канал

Вызов функции: четыре шага



§ 7.1–7.5

Материалы

Telegram-канал

Второй запрос несёт в модель всю историю

```
"role": "user"
"content": [{"text": "Поместятся ли 28 студентов в A-101?"}]
```

```
"role": "assistant"
"content": [{"function_call": {"name": "get_room_info",
 "arguments": {"room": "A-101"}}}],
"tools_state_id": "01a0f48d-…"
```

```
"role": "tool"
"content": [{"function_result": {"name": "get_room_info",
 "result": {"room": "A-101", "capacity": 24, "projector": true}}}]
```

+ `tools` — в каждом запросе: сервер историю не хранит



§ 7.5

Материалы

Telegram-канал

Режимы вызова и проверка

Команда Сбера

`tool_config` — кто решает, вызывать ли функцию

не задан · `{"mode": "auto"}`	модель решает сама
`{"mode": "none"}`	функции не вызываются
`{"mode": "forced", "function_name": "get_room_info"}`	вызов обязателен
`{"mode": "forsed"}`	без ошибки → как `auto`

`execute_tool` — проверка перед исполнением



`forced` на «Привет!» → `get_room_info` с придуманным аргументом: пустая строка или `"A-101"`

Проверка ловит неверную форму вызова. Правдоподобный выдуманный аргумент она пропустит.

§ 7.6

Материалы

Telegram-канал

Несколько вызовов: цикл



вопрос: «Сравни вместимость A-101 и B-202»

шаг 1 `get_room_info({"room": "A-101"})` → 24 места

шаг 2 `get_room_info({"room": "B-202"})` → 40 мест

шаг 3 «A-101 — 24 человека, B-202 — 40»

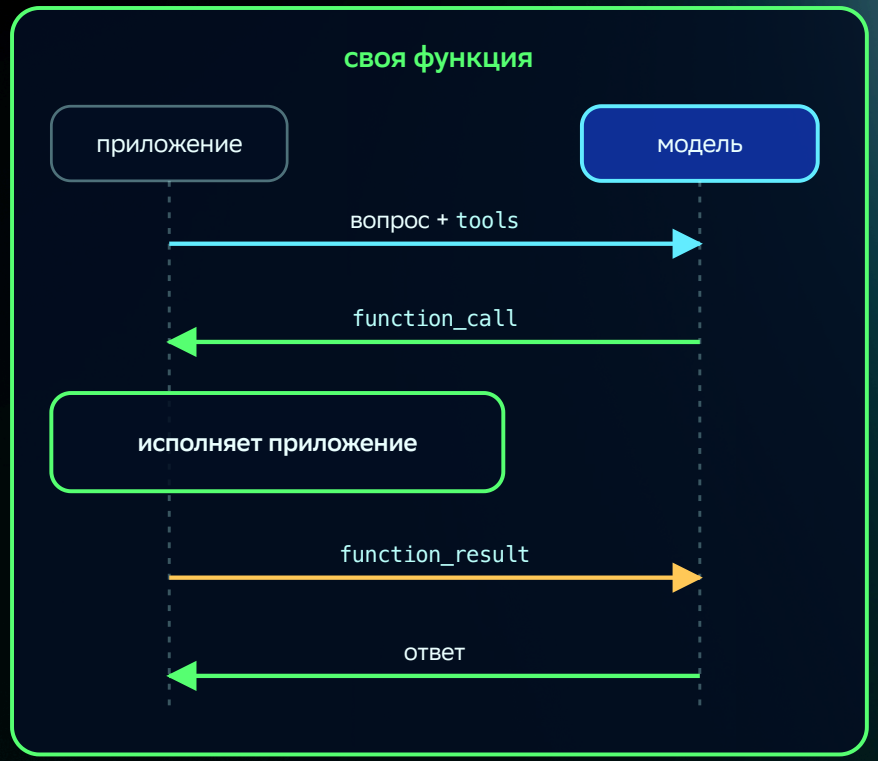
этот цикл — ядро любого агента

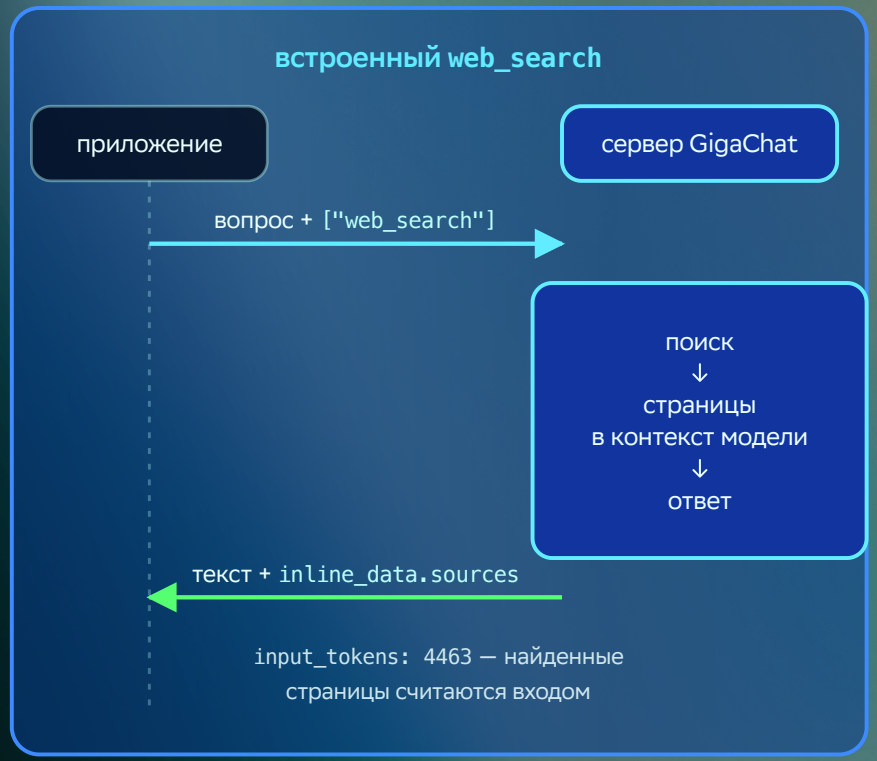
§ 7.7

Материалы

Telegram-канал

Встроенный инструмент

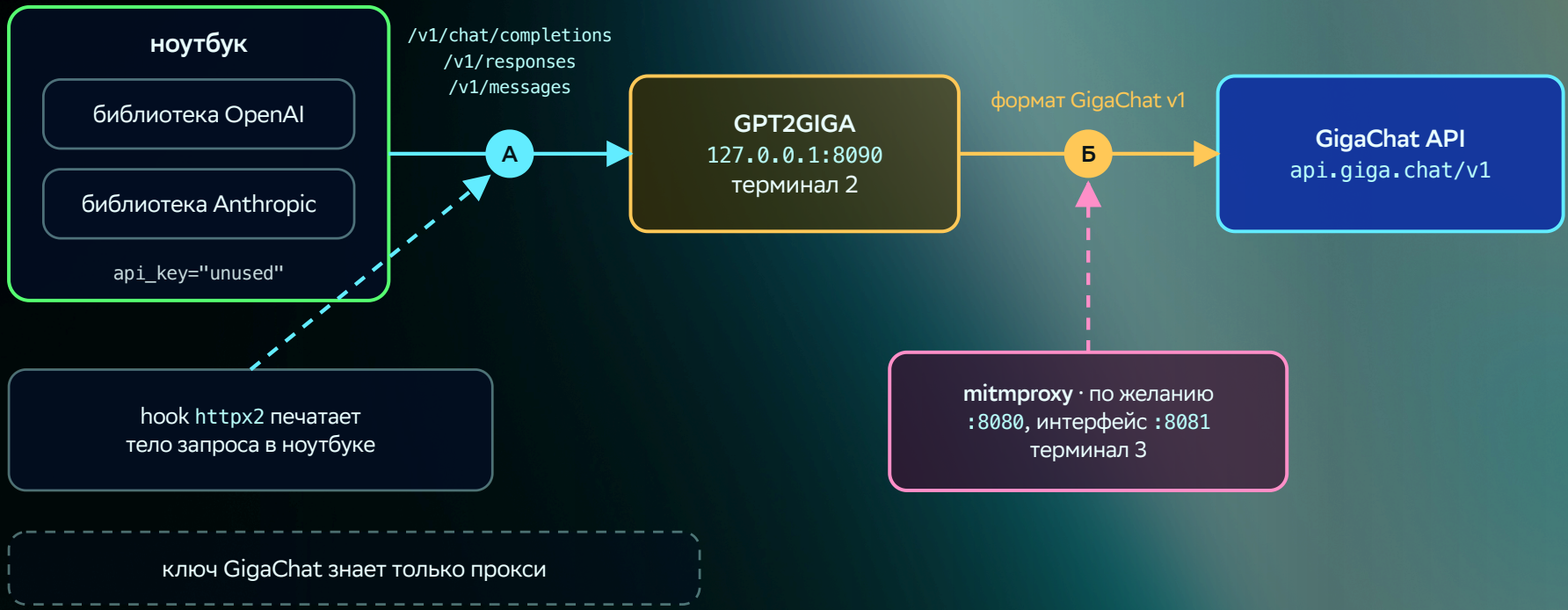




Материалы

Telegram-канал

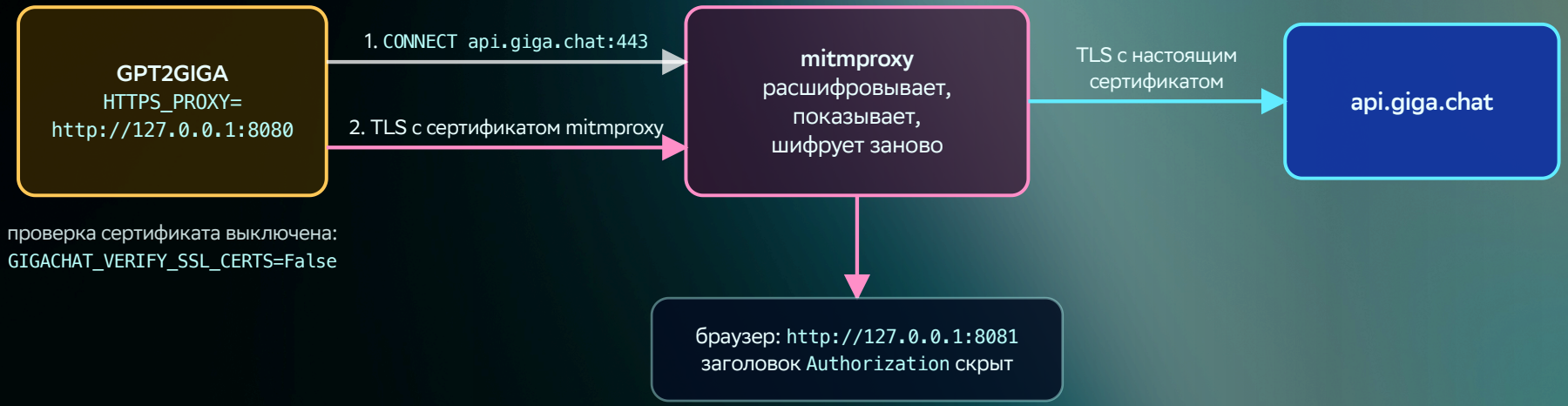
GPT2GIGA: GigaChat в форматах OpenAI и Anthropic



Материалы

Telegram-канал

Как mitmproxy видит HTTPS



запрос токена `ngw.devices.sberbank.ru` идёт через mitmproxy туннелем, без расшифровки

`--ignore-hosts` в `tools/start_mitmproxy.py`

§ 9.2–9.3 · 13

Материалы

Telegram-канал

Один вызов — три формата

описание функции · вызов · результат · связь вызова и результата

→ в запросе · ← в ответе

CHAT COMPLETIONS

```
→ messages: [{"role": "system", …}]
→ tools: [{"type": "function",
    "function": {name, parameters}}]
← message.tool_calls: [{
    "id": "01a0…",
    "function": {"name": …,
      "arguments": "{\"room\": …}"}}]
→ {"role": "tool",
   "tool_call_id": "01a0…",
   "content": "{…24…}"}
```

аргументы — JSON-строка

RESPONSES

```
→ instructions: "…"
→ tools: [{"type": "function",
    name, parameters}]
← output: [{"type": "function_call",
    "call_id": "01a0…",
    name, "arguments": "{…}"}]
→ input += [{
    "type": "function_call_output",
    "call_id": "01a0…",
    "output": "{…24…}"}]
```

аргументы — JSON-строка

MESSAGES

```
→ system: "…", max_tokens: 512
→ tools: [{name, input_schema}]
← content: [{"type": "tool_use",
    "id": "01a0…",
    name, "input": {"room": …}}]
→ {"role": "user", "content": [{
    "type": "tool_result",
    "tool_use_id": "01a0…",
    "content": "{…24…}"}]}
```

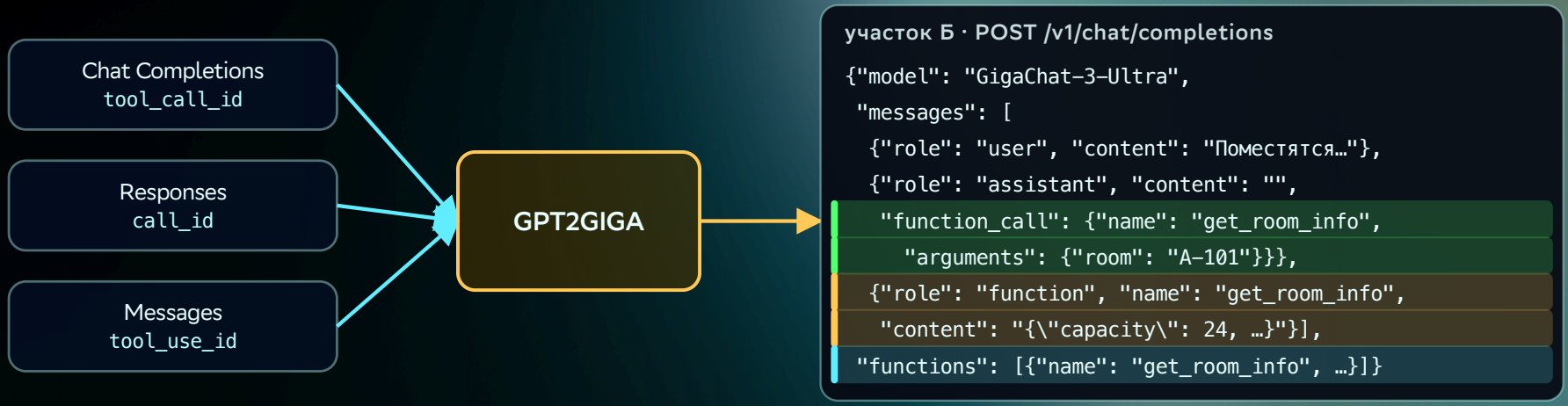
аргументы — объект

§ 10–12

Материалы

Telegram-канал

Что прокси отправляет в GigaChat



остаются у прокси:

- ✕ ID вызова
- ✕ `tool_choice: "required"`
- ✕ `store · previous_response_id`

ни одного ID вызова — связь по имени функции

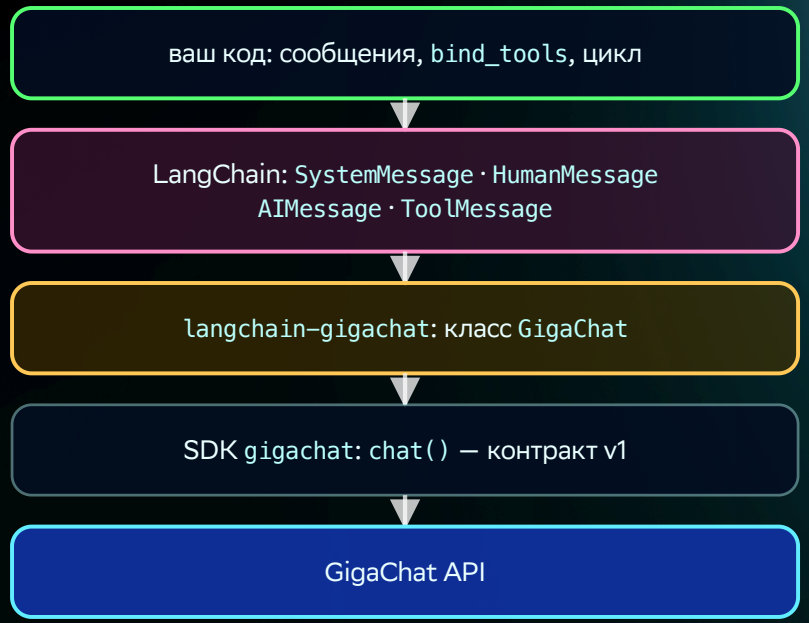
строка-результат → `{"result": "…"}`: GigaChat принимает только JSON

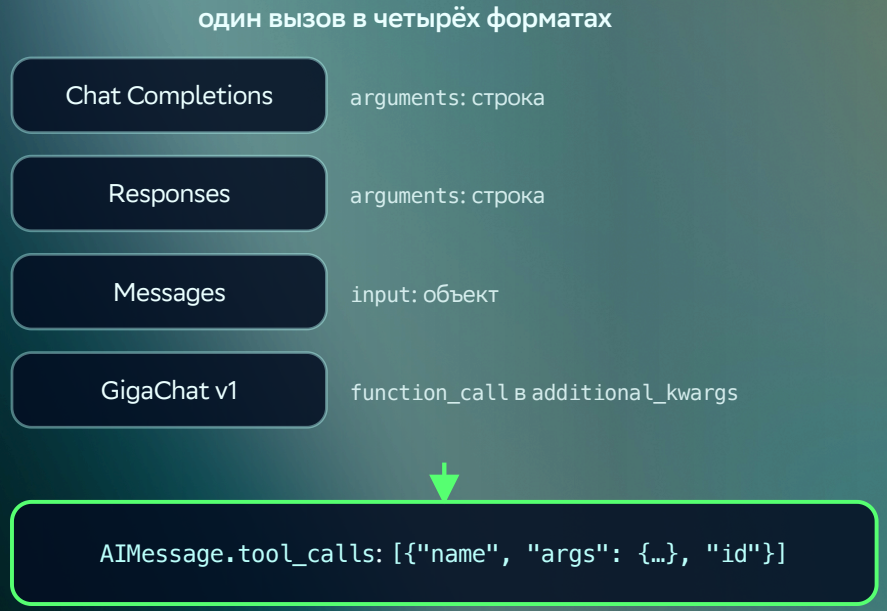
три формата на входе → один запрос GigaChat v1 на выходе

Материалы

Telegram-канал

LangChain: общие объекты вместо форматов





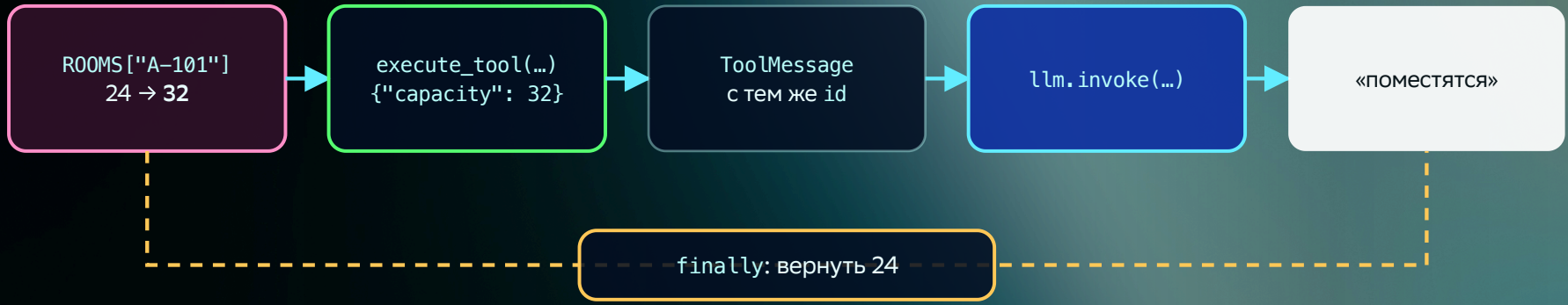
остаётся приложению: исполнить и проверить вызов,
вести историю, учитывать особенности провайдера

Материалы

Telegram-канал

Практика: новый факт при прежнем вопросе

вопрос и инструкция прежние — меняется только факт из справочника



данные инструмента попадают прямо в контекст модели и определяют ответ

§ 15.1

Материалы

Telegram-канал

Кто продолжит цикл

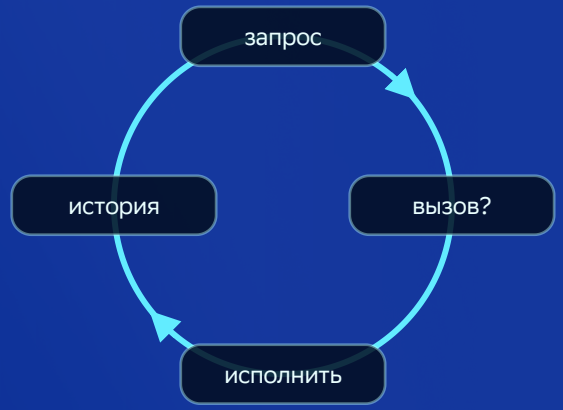
Команда Сбера

сегодня: цикл пишете вы



следующая пара: цикл ведёт агент

`create_agent` · middleware · Deep Agents



§ 16

Материалы

Telegram-канал

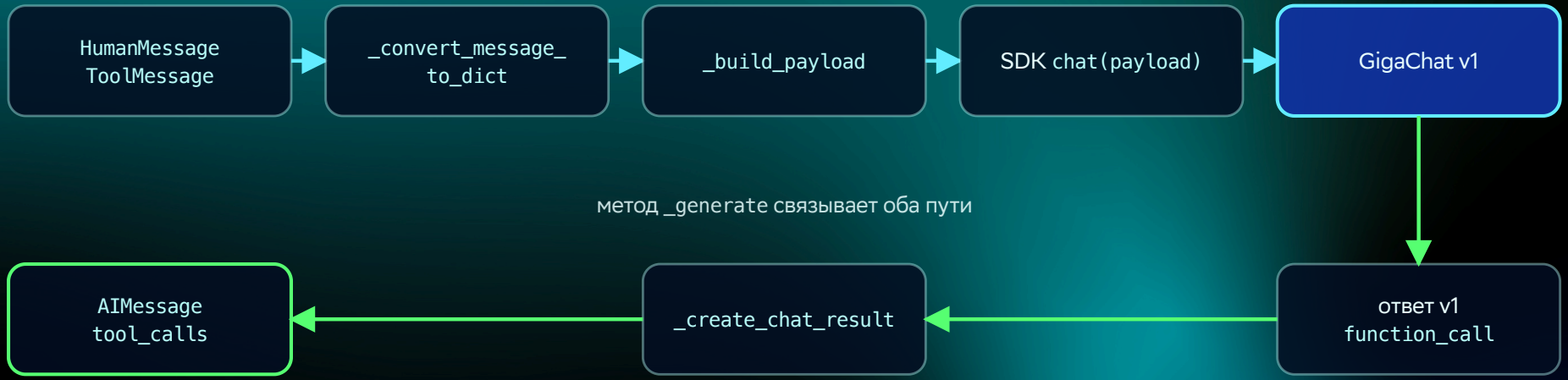
Команда Сбера

Вопросы

Материалы

Telegram-канал

Приложение: путь через адаптер LangChain




```
print(inspect.getsource(langchain_gigachat.GigaChat._generate))
```

Приложение

Материалы

Telegram-канал

Команда Сбера

Источники

Все ссылки по темам — в `SOURCES.md` в репозитории материалов.

- GigaChat API: developers.sber.ru/docs/ru/gigachat
- SDK: github.com/ai-forever/gigachat
- GPT2GIGA: github.com/ai-forever/gpt2giga
- LangChain: docs.langchain.com
- mitmproxy: docs.mitmproxy.org

Материалы

Telegram-канал